Aplicación de Docker en el proyecto Safari Kids

Contenerización del frontend, backend y base de datos con Docker Compose

Safari Kids · Informe técnico · Julio de 2026

1. Introducción

Docker es una plataforma de contenedores: permite empaquetar una aplicación junto con todo lo que necesita para ejecutarse (runtime, librerías, configuración) en una unidad aislada y portable llamada **contenedor**. En Safari Kids adoptamos Docker para que todo el sistema —frontend en React, backend en Spring Boot y base de datos PostgreSQL— se levante con **un solo comando** (`docker compose up -d`), sin que cada integrante del equipo tenga que instalar Java, Node o PostgreSQL en su máquina, y garantizando que el entorno local se comporte igual que producción (Railway).

2. Arquitectura de contenedores

El sistema se divide en **tres contenedores**, uno por responsabilidad, orquestados por Docker Compose y conectados entre sí mediante una red interna que Docker crea automáticamente:



Dentro de la red interna los contenedores se encuentran **por nombre de servicio**: el frontend reenvía las peticiones a `http://backend:8080` y el backend se conecta a `jdbc:postgresql://db:5432/safari_kids`. La base de datos no expone ningún puerto al exterior: solo el backend puede alcanzarla, lo que añade una capa de seguridad.

3. Orquestación con Docker Compose

El archivo `docker-compose.yml`, en la raíz del proyecto, define los tres servicios y cómo se relacionan. Sus decisiones clave son:

Elemento	Qué hace	Por qué
`healthcheck` en `db`	Ejecuta `pg_isready` cada 5 s hasta que PostgreSQL acepte conexiones.	Evita que el backend arranque antes de que la base de datos esté lista.
`depends_on: condition: service_healthy`	El backend solo inicia cuando el healthcheck de la BD pasa.	Sin esto, Spring Boot fallaría al no poder conectarse en el arranque.
Variables de entorno	Credenciales de BD, CORS y URL del frontend se inyectan al backend.	La misma imagen sirve para local y producción; solo cambia la configuración.
Volumen `db_data`	Monta `/var/lib/postgresql/data` en un volumen de Docker.	Los datos sobreviven aunque el contenedor se elimine o recree.
Mapeo de puertos	Frontend `3000:80` y backend `8081:8080`.	El 8080 del host suele estar ocupado por el backend en modo desarrollo; dentro de la red Docker sigue siendo `backend:8080`.

Extracto del archivo:

```
services:
  db:
    image: postgres:16-alpine
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U safari -d safari_kids"]

  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/safari_kids
    ports:
      - "8081:8080"

  frontend:
    build: ./frontend
    depends_on:
      - backend
    ports:
      - "3000:80"
```

4. Imagen del backend (multi-stage build)

El `backend/Dockerfile` usa una **compilación en dos etapas**: la primera contiene Maven y el JDK completo para compilar; la segunda solo el JRE de Alpine con el JAR resultante. Así la imagen final pesa ~200 MB en lugar de los ~800 MB que ocuparía con todo el toolchain de compilación.

```
# Etapa 1: compilación con Maven
FROM maven:3.9-eclipse-temurin-17 AS build
COPY pom.xml .
RUN mvn dependency:go-offline -B      # cachea las dependencias
COPY src ./src
RUN mvn package -DskipTests -B

# Etapa 2: solo el JRE + el JAR
FROM eclipse-temurin:17-jre-alpine
COPY --from=build /app/target/safari-kids-backend-1.0.0.jar app.jar
ENTRYPOINT ["java", "-Xmx256m", "-Xss512k", "-XX:MaxMetaspaceSize=150m",
            "-XX:ReservedCodeCacheSize=64m", "-XX:+UseSerialGC", "-jar", "app.jar"]
```

- **Caché de dependencias:** se copia primero solo el `pom.xml` y se descargan las dependencias. Si el `pom.xml` no cambia, Docker reutiliza esa capa y los rebuilds tardan segundos en vez de minutos.
- **Paridad con producción:** los límites de memoria de la JVM (`-Xmx256m`, etc.) son los mismos que usamos en Railway (contenedor de 512 MB), de modo que un problema de memoria se detecta en local antes de llegar a producción.

5. Imagen del frontend (build de Vite + Nginx)

El `frontend/Dockerfile` también es multi-stage: Node 20 compila la aplicación con `npm run build` y el resultado (archivos estáticos de `dist/`) se copia a una imagen de **Nginx**, que es quien realmente atiende a los usuarios. Node no viaja en la imagen final.

```
# Etapa 1: build de Vite
FROM node:20-alpine AS build
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Etapa 2: servir los estáticos con Nginx
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

La configuración de Nginx (`frontend/nginx.conf`) cumple dos funciones:

- **Proxy inverso:** toda petición a `/api/` se reenvía a `http://backend:8080`. Es el mismo papel que cumple el proxy de Vite en desarrollo, por lo que el código del frontend no cambia entre entornos y no hay problemas de CORS.
- **Soporte de SPA:** con `try_files $uri $uri/ /index.html`, cualquier ruta desconocida devuelve `index.html` y React Router resuelve la navegación, evitando errores 404 al recargar la página.

6. Persistencia de datos

Los contenedores son efímeros: al recrearlos, su sistema de archivos se pierde. Por eso el servicio `db` monta el volumen `db_data` sobre el directorio de datos de PostgreSQL. Con `docker compose down` los datos se conservan; solo `docker compose down -v` los elimina deliberadamente (útil para reiniciar la BD desde cero, ya que las migraciones de Flyway del backend vuelven a crear el esquema al arrancar).

7. Comandos de uso diario

Comando	Efecto
`docker compose up -d`	Construye (si hace falta) y levanta los tres contenedores en segundo plano.
`docker compose up -d --build`	Fuerza la reconstrucción de las imágenes tras cambiar el código.
`docker compose ps`	Muestra el estado de los contenedores.
`docker compose logs -f backend`	Sigue los logs de un servicio en vivo.
`docker compose down`	Detiene y elimina los contenedores (los datos de la BD se conservan).
`docker compose down -v`	Igual, pero borrando también el volumen de la base de datos.

Con el stack arriba, la aplicación queda disponible en:

- Frontend: `http://localhost:3000`
- API del backend (acceso directo para pruebas): `http://localhost:8081/api`

8. Beneficios obtenidos

- **Entorno reproducible:** cualquier integrante levanta el sistema completo con un comando, sin instalar Java 17, Node 20 ni PostgreSQL.
- **Paridad con producción:** misma base de datos (PostgreSQL), mismos límites de memoria de la JVM y mismo esquema de proxy que en Railway.
- **Aislamiento:** cada servicio corre en su propio contenedor con sus propias dependencias; la BD ni siquiera es accesible desde fuera de la red interna.

- **Imágenes ligeras:** los builds multi-stage dejan fuera los toolchains de compilación (Maven, JDK, Node), reduciendo el tamaño y la superficie de ataque.
- **Builds rápidos:** el cacheo de capas de Docker hace que reconstruir tras un cambio de código tarde segundos.

> **Nota:** las credenciales que aparecen en `docker-compose.yml` son únicamente para el entorno local. En producción (Railway) la configuración se inyecta mediante variables de entorno del servicio y no está versionada en el repositorio.